Generative Models

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)
x is data, y is label

Goal: Learn a function to map x -> y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)
x is data, y is label

Goal: Learn a function to map x -> y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

Data: x
Just data, no labels!

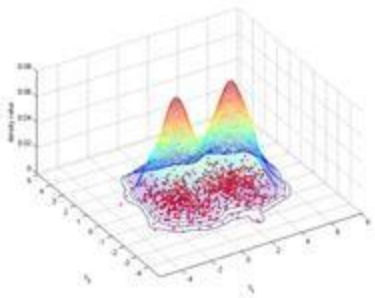
Goal: Learn hidden structure in data

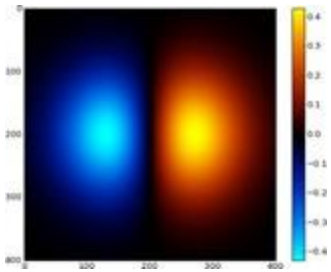
Examples: Clustering, dimensionality reduction, density estimation, etc.

Supervised vs Unsupervised Learning

Figure copyright Ian Goodfellow, 2016. Reproduced with permission.

1-d density estimation





2-d density estimation

Modeling p(x)

2-d density images left and right are CC0 public domain

Unsupervised Learning

Data: x
Just data, no labels!

Goal: Learn hidden structure in data

Examples: Clustering, dimensionality reduction, density estimation, etc.

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

Data: x

Label: y

Cat

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

Data: x

Label: y

Cat

Probability Recap:

Density Function
p(x) assigns a positive number to each possible x; higher numbers mean x is more likely.

Density functions are **normalized:**

$$\int_X p(x)\, dx = 1$$

Different values of x **compete** for density

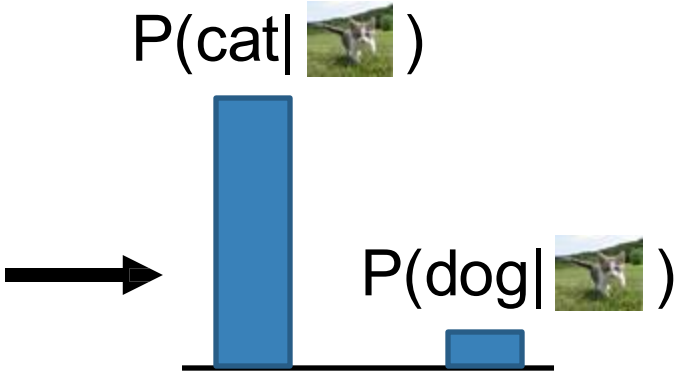
Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Data: x

Label: y

Cat

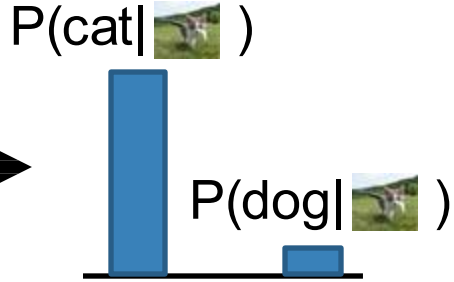


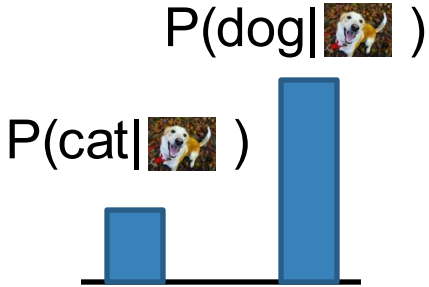
$\int_X p(x)\, dx = 1$

Different values of x **compete** for density

Generative vs Discriminative Models

Discriminative Model:
Learn a probability distribution p(y|x)



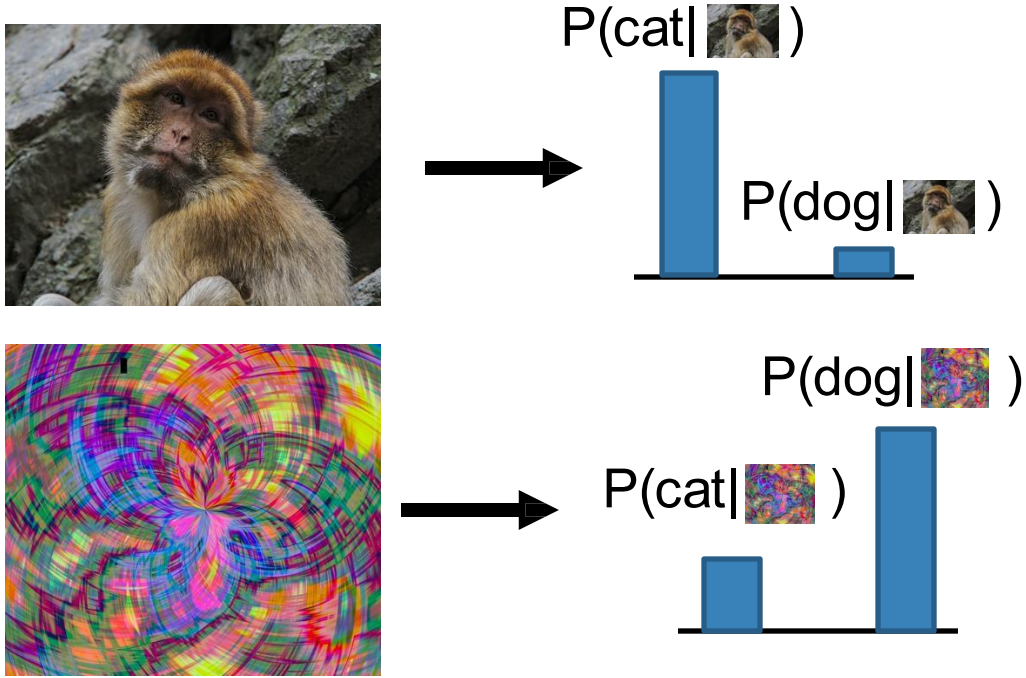


Possible **labels** for each image compete for probability.
No competition between **images**

Monkey image is CC0 Public Domain
Abstract image is free to use under the Pixabay license

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)



No way to handle unreasonable inputs; must give a label distribution for all possible inputs

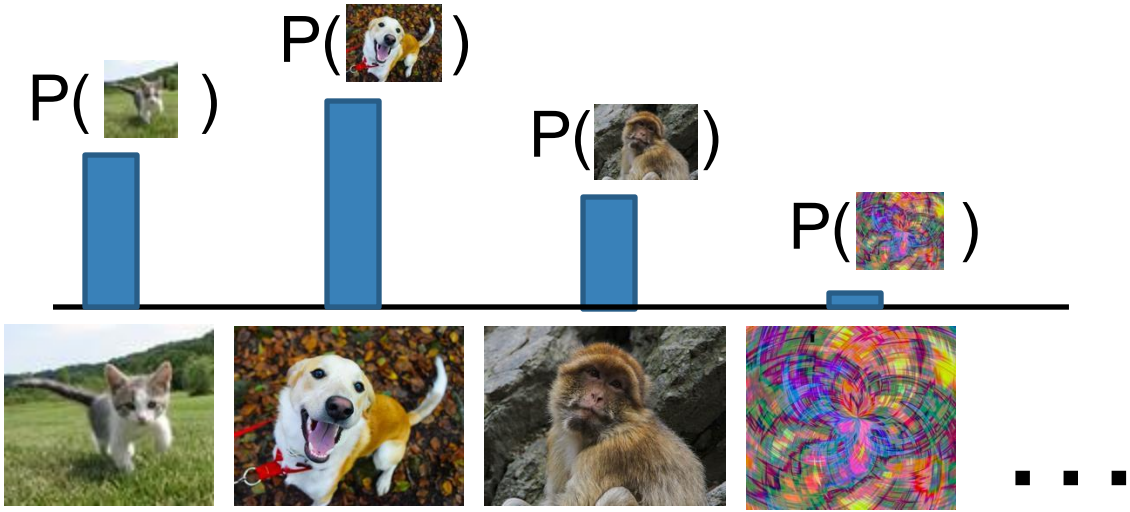
Cat image is CC0 public domain
Dog image is CC0 Public Domain
Monkey image is CC0 Public Domain
Abstract image is free to use under the Pixabay license

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)



All possible **images** compete for probability mass

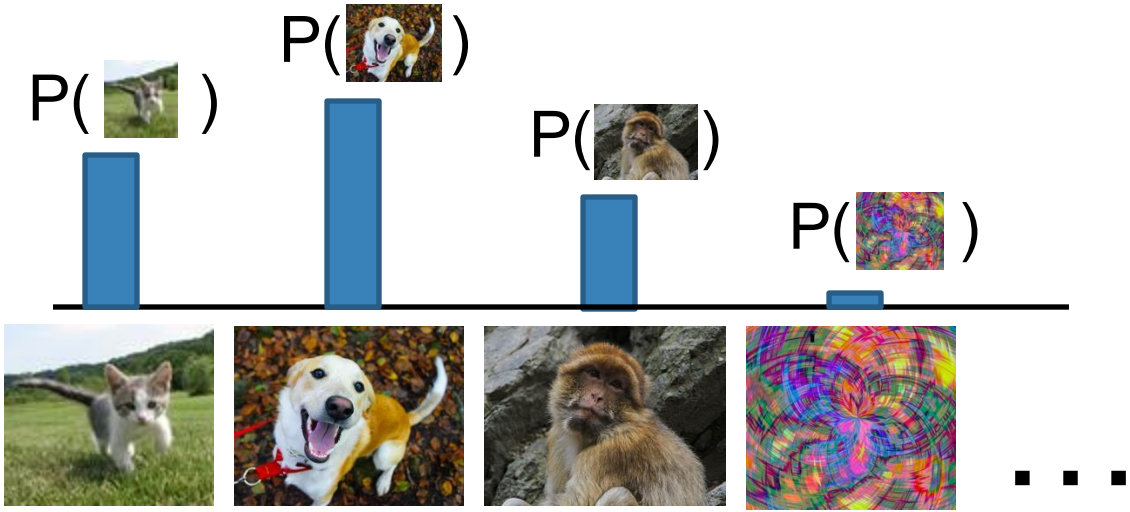
Cat image is CC0 public domain
Dog image is CC0 Public Domain
Monkey image is CC0 Public Domain
Abstract image is free to use under the Pixabay license

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)



All possible **images** compete for probability mass

Requires deep understanding: Is a dog more likely to sit or stand? Is a 3-legged dog more likely than a 3-armed monkey?

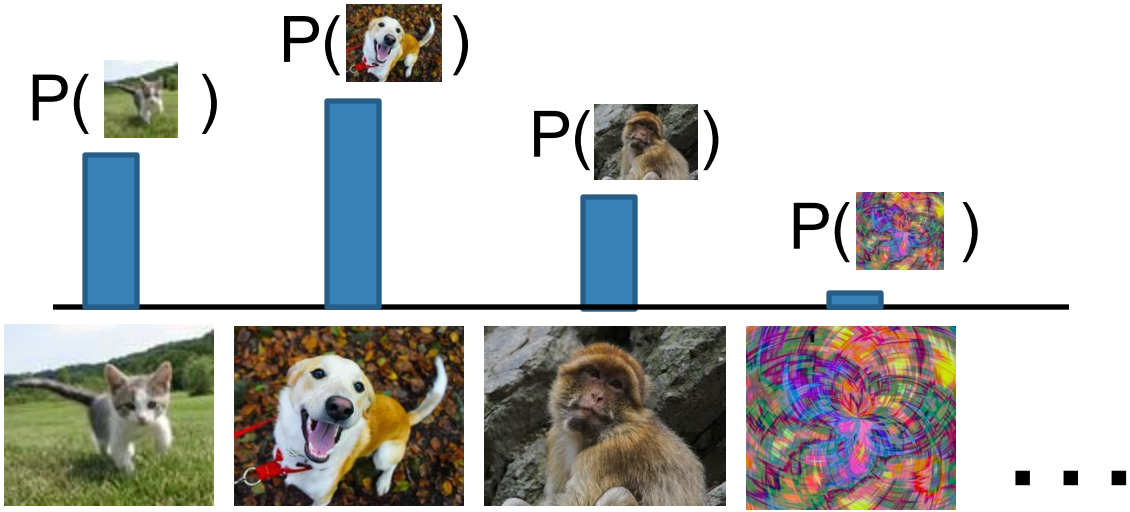
Cat image is CC0 public domain
Dog image is CC0 Public Domain
Monkey image is CC0 Public Domain
Abstract image is free to use under the Pixabay license

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

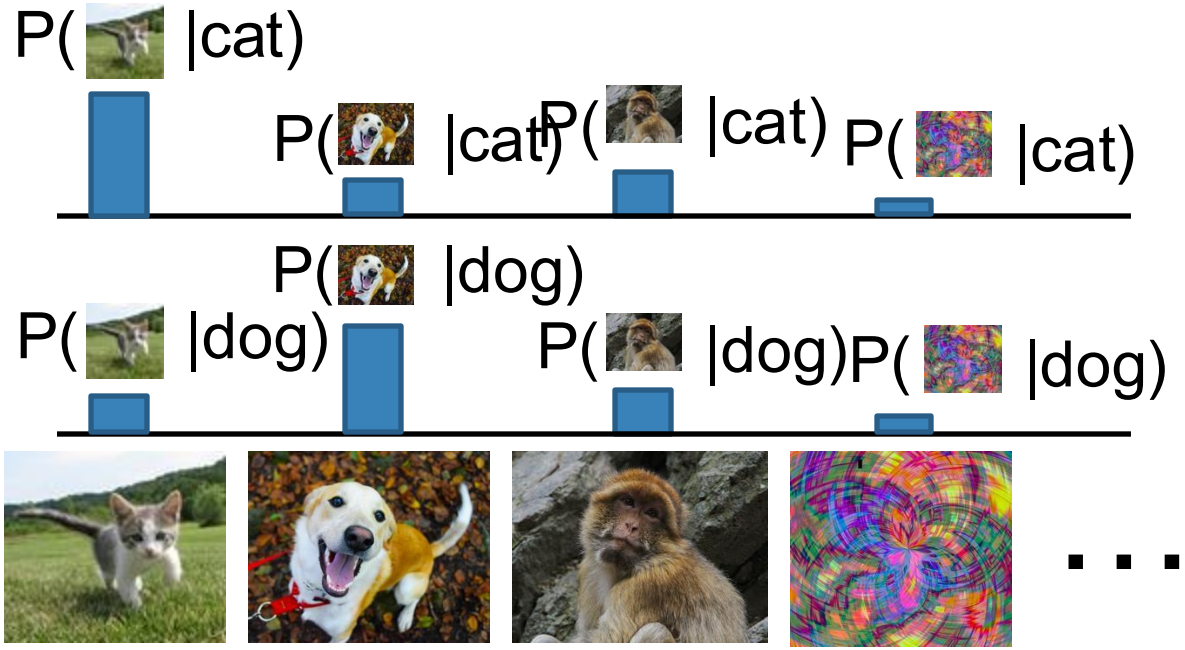


All possible **images** compete for probability mass

Model can “reject” unreasonable inputs by giving them small probability mass

Cat image is CC0 public domain
Dog image is CC0 Public Domain
Monkey image is CC0 Public Domain
Abstract image is free to use under the Pixabay license

Generative vs Discriminative Models



Conditional Generative Model: Learn p(x|y)

Each possible **label** induces a competition across all possible **images**

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

Recall **Bayes' Rule**:

$$P(x \mid y) = \frac{P(y \mid x)}{P(y)} P(x)$$

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

Recall **Bayes' Rule**:

Discriminative Model

$$P(x \mid y) = \frac{P(y \mid x)}{P(y)} P(x)$$

Conditional Generative Model

Prior over labels

(Unconditional) Generative Model

We can build a conditional generative model from other components … but not common in practice

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution $p(y|x)$

→ Assign labels to data
Feature learning (with labels)

Generative Model: Learn a probability distribution $p(x)$

→ Detect outliers
Feature learning (without labels)
Sample to generate new data

Conditional Generative Model: Learn $p(x|y)$

→ Assign labels while rejecting outliers
Sample to generate data from labels

Generative vs Discriminative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

”Generative models” means either of these; conditional generative models are most common in practice

Why Generative Models?

Modeling ambiguity: If there are many possible outputs x for an input y, we want to model P(x | y)

Language Modeling: Produce output text x from input text y

Write me a short rhyming poem about generative models

→

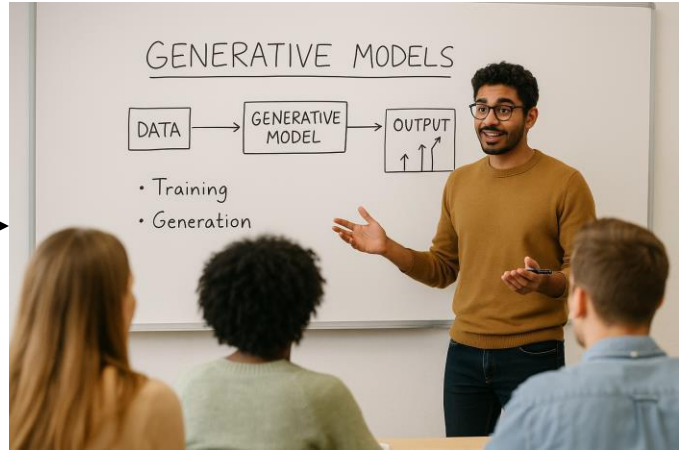
*They sample from a learned **P**,*
A distribution—structured, free.
Each token comes conditionally,
On all the ones that used to be.

Why Generative Models?

Modeling ambiguity: If there are many possible outputs x for an input y, we want to model P(x | y)

Text to Image: Produce output image x from input text y

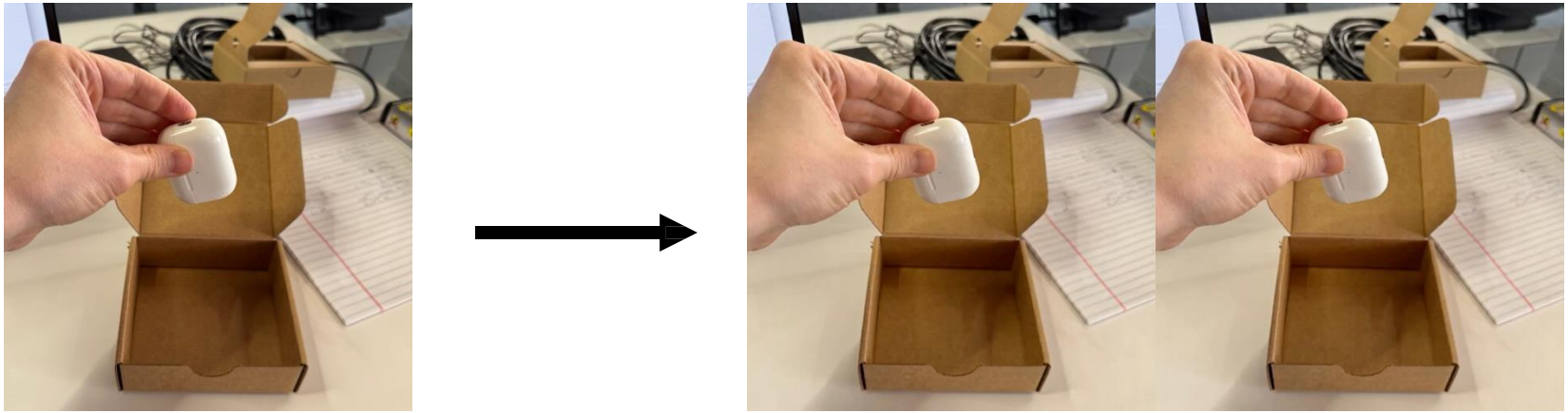
Make me an image showing a person teaching a class on generative models in front of a whiteboard



Why Generative Models?

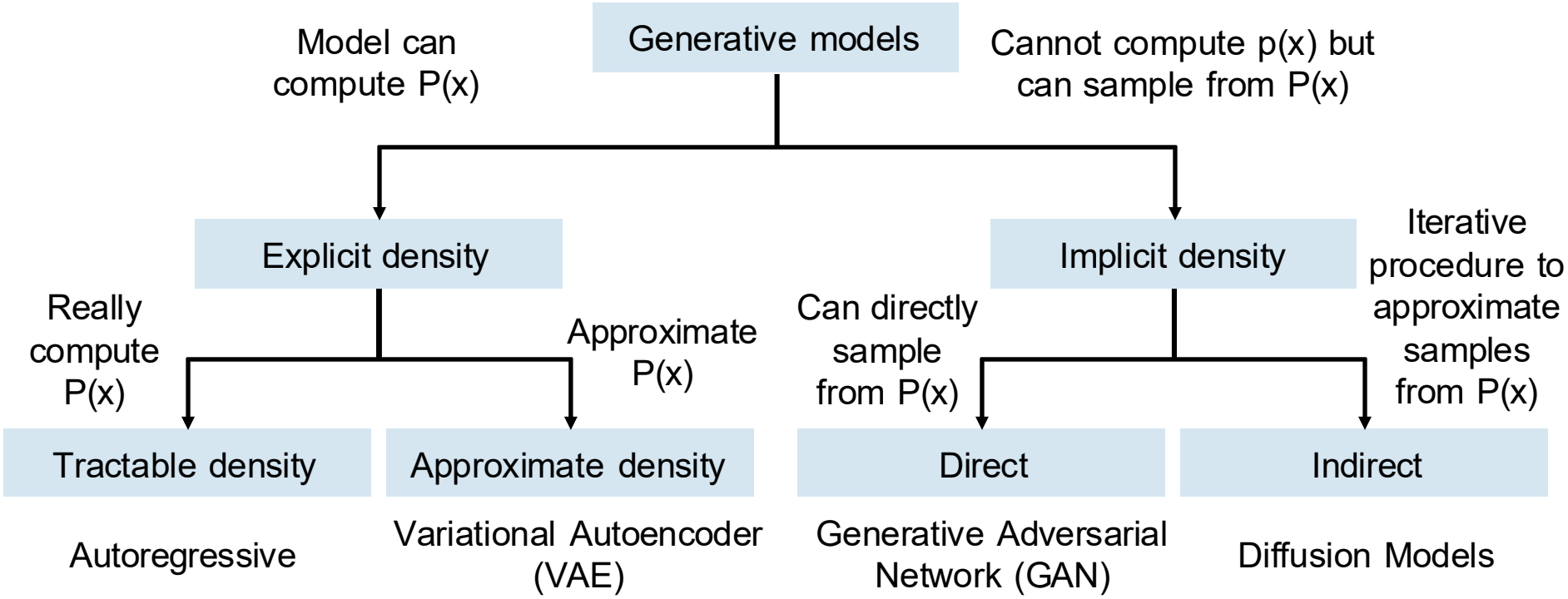
Modeling ambiguity: If there are many possible outputs x for an input y, we want to model P(x | y)

Image to Video: What happens next?



Taxonomy of Generative Models

Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017



Autoregressive Models

Maximum Likelihood Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \ldots x^{(N)}$, train the model by solving:

$$W^* = \arg\max_{\mathrm{W}} \prod_i p(x^{(i)})$$

Maximize probability of training data
(Maximum likelihood estimation)

$$= \arg\max_{W} \Sigma_i \ \log p(x^{(i)})$$

Log trick: Swap product and sum

$$= \arg\max_{W} \Sigma_i \ \log f(x^{(i)}, W)$$

This is our loss function.
maximize it with gradient descent

Autoregressive Models

Goal: Write down an explicit function for $p\,(x) = f(x, W)$

Assume x is a sequence: $x = (x_1, x_2, \ldots, x_T)$

Autoregressive Models

Goal: Write down an explicit function for $p\,(x) = f(x, W)$

Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$

Use the chain rule of probability:

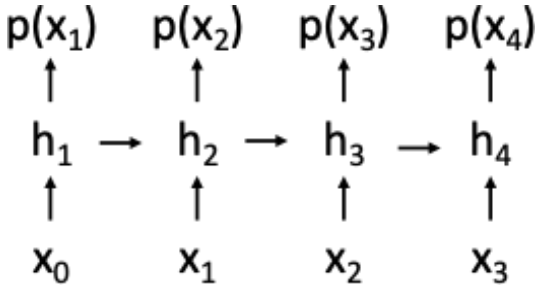
$$
\begin{aligned}
p(x) &= p(x_1, x_2, x_3, \dots, x_T\ ) \\
&= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \dots \\
&= \prod_{t=1}^{T} p(x_t \mid x_1, \dots, x_{t-1})
\end{aligned}
$$

Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$

We have already seen this!



Language modeling with RNN

Use the chain rule of probability:

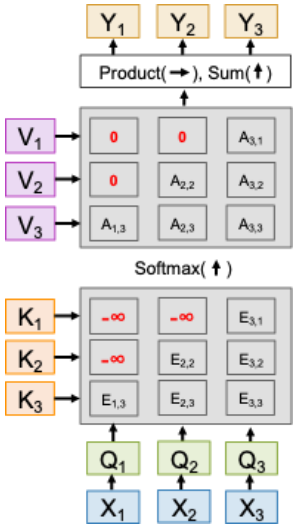
$$
\begin{aligned}
p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\
&= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \dots \\
&= \prod_{t=1}^{T} p(x_t \mid x_1, \dots, x_{t-1})
\end{aligned}
$$

LLMs are Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$

Language modeling with masked Transformer



Use the chain rule of probability:

$$
\begin{aligned}
p(x) &= p(x_1, x_2, x_3, \dots, x_T) \\
&= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \dots \\
&= \prod_{t=1}^{T} p(x_t \mid x_1, \dots, x_{t-1})
\end{aligned}
$$

Autoregressive Models of Images

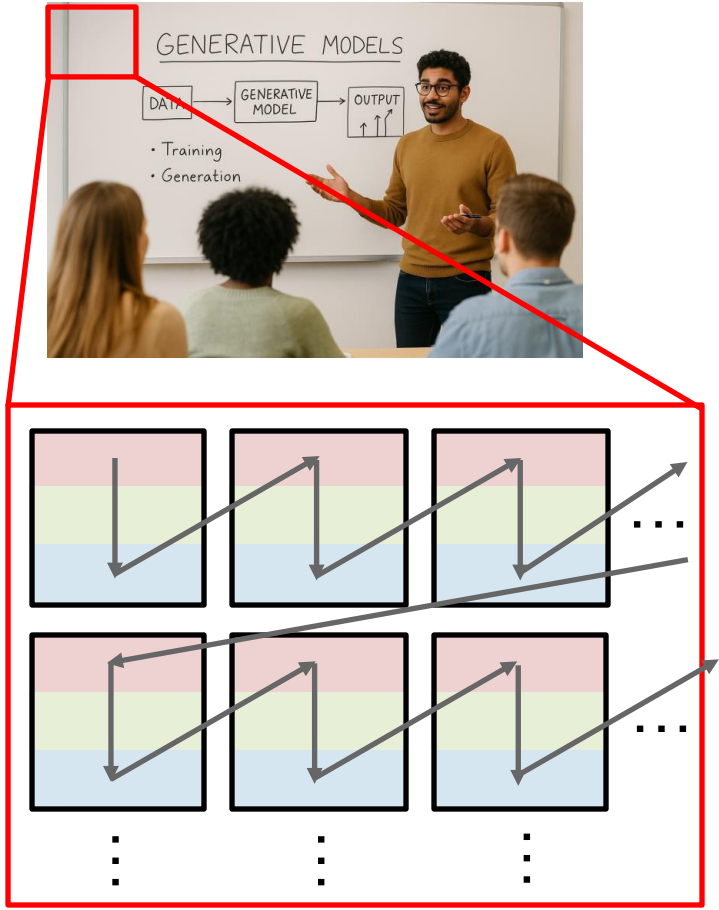
Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values [0…255]

Model with an RNN or Transformer

Problem: Too expensive. 1024x1024 image is a sequence of 3M subpixels

Solution (jumping ahead): Model as sequence of tiles, not sequence of subpixels



Variational Autoencoders (VAEs)

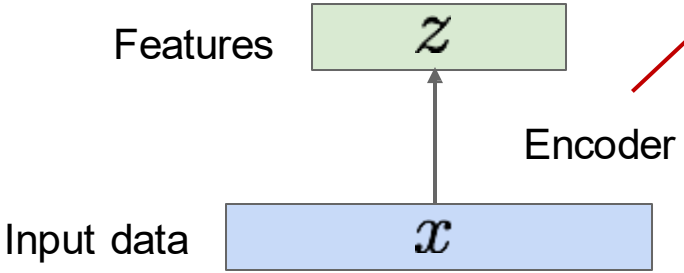
Variational Autoencoders (VAEs)

(Non-Variational) Autoencoders

Idea: Unsupervised method for learning to extract features z from inputs x, without labels

Features should extract useful information
(object identity, appearance, scene type, etc)
that can be used for downstream tasks

Encoder can be MLP,
CNN, Transformer, …

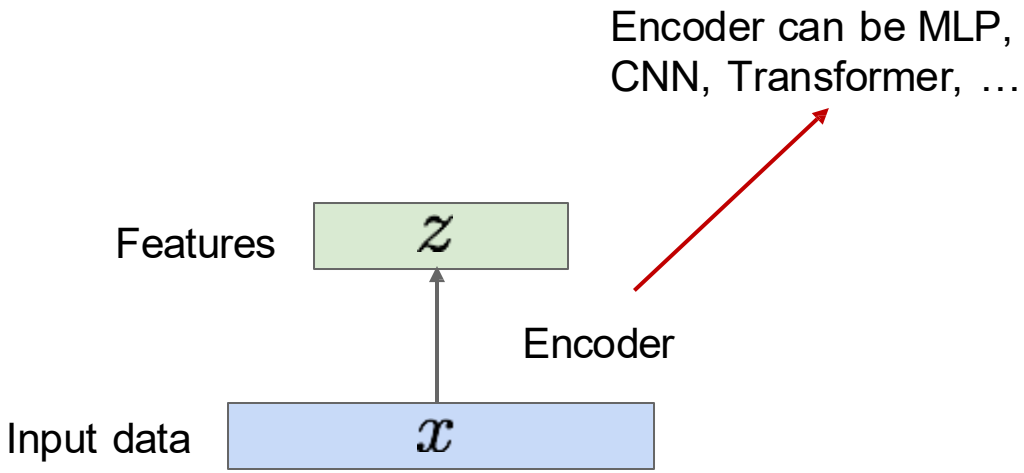


Input Data

(Non-Variational) Autoencoders

Problem: How can we learn without labels?

Features should extract useful information (object identity, appearance, scene type, etc) that can be used for downstream tasks



Input Data

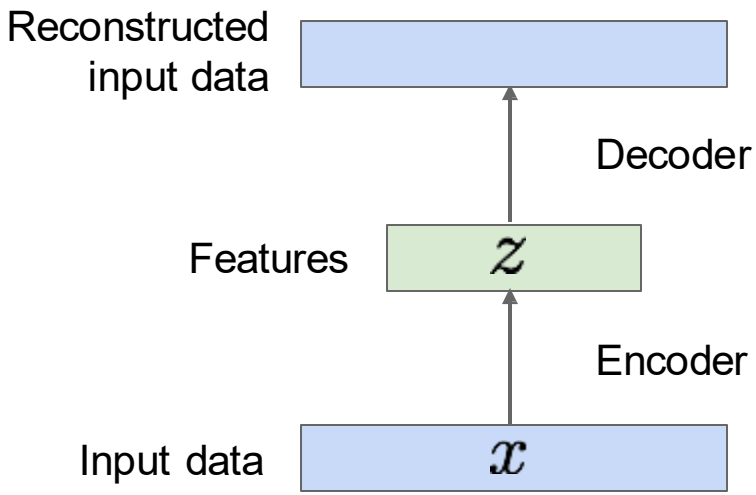
(Non-Variational) Autoencoders

"Autoencoding" = Encoding yourself

Problem: How can we learn without labels?

Solution: Reconstruct the input data with a decoder.

Decoder can be MLP, CNN, Transformer, …

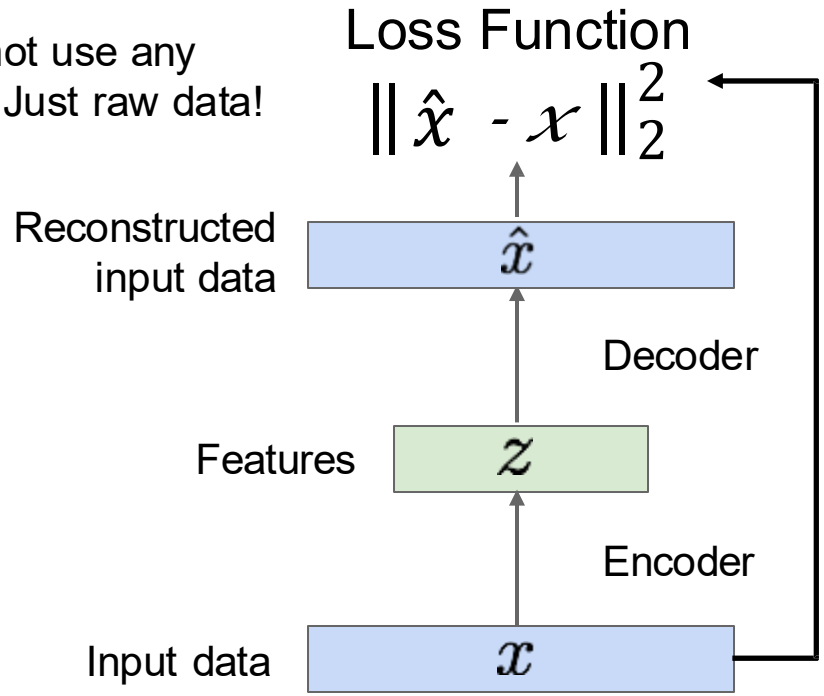


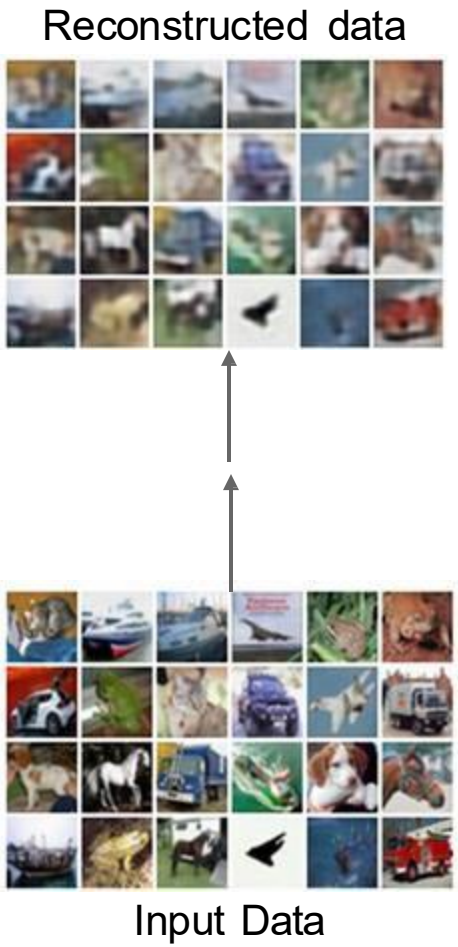
Input Data

(Non-Variational) Autoencoders

Loss: L2 distance between input and reconstructed data.

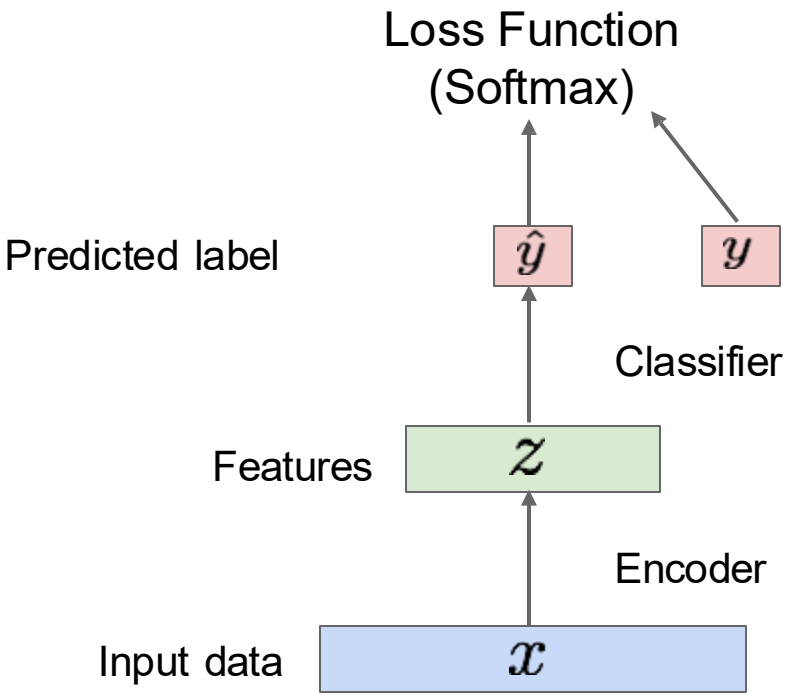
Does not use any labels! Just raw data!

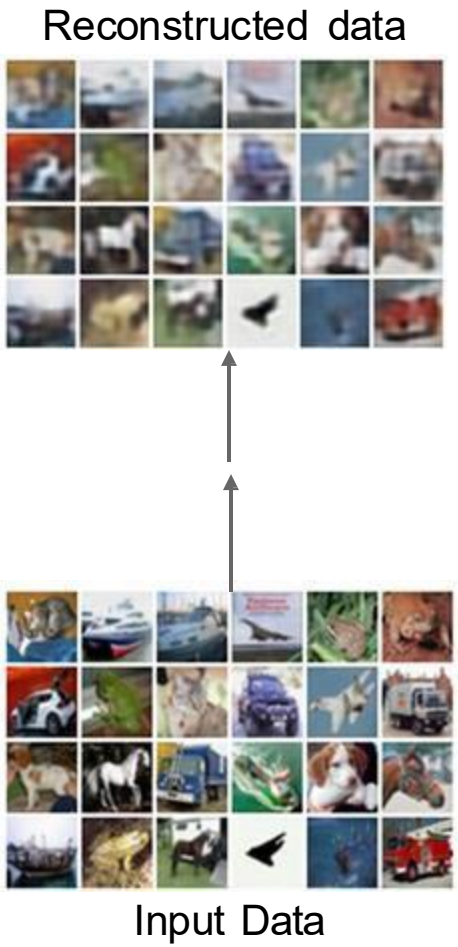




(Non-Variational) Autoencoders

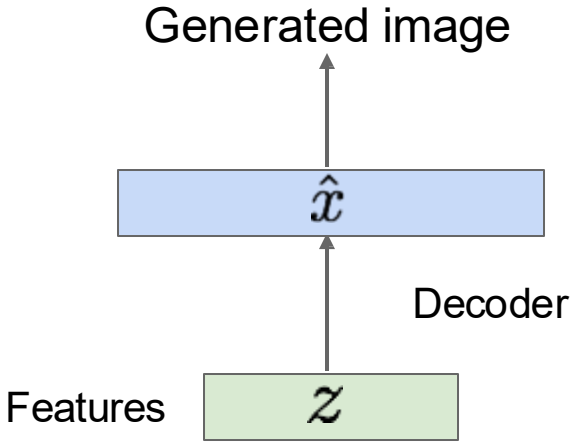
After training, can use encoder for downstream tasks





(Non-Variational) Autoencoders

If we could generate new z, could use the decoder to generate images



Problem: Generating new z is not any easier than generating new x

Solution: What if we force all z to come from a known distribution?

Variational Autoencoders (VAEs)

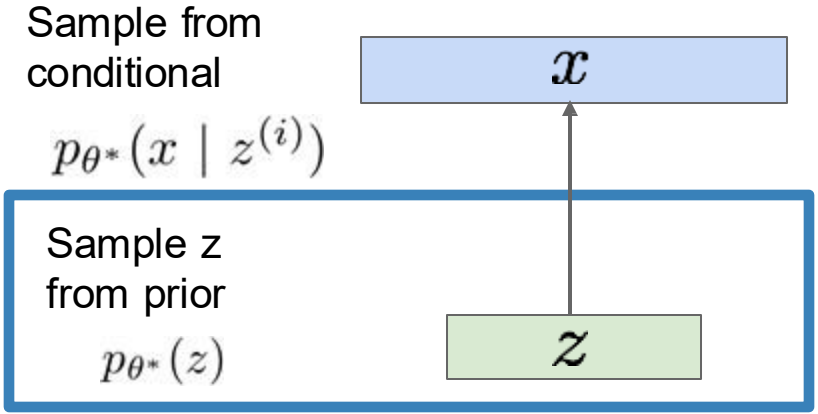
Kingma and Welling, Auto-Encoding Variational Beyes, ICLR 2014

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

Intuition: x is an image, **z** is latent factors used to generate **x:** attributes, orientation, etc.

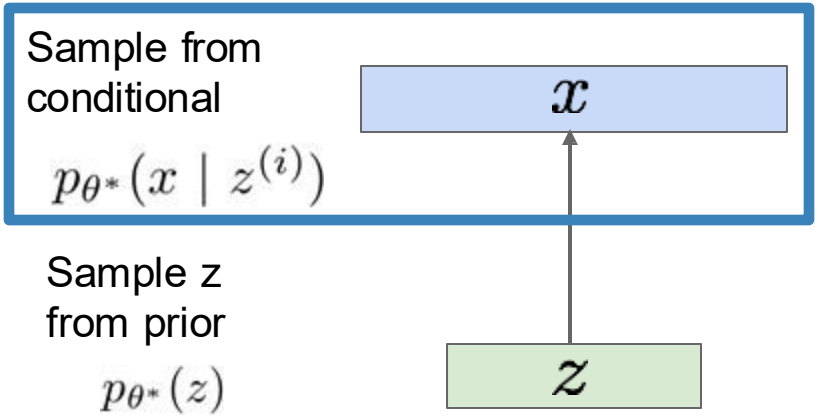
Assume simple prior p(z), e.g. Gaussian

Variational Autoencoders

Probabilistic spin on autoencoders:

1. Learn latent features z from raw data
2. Sample from the model to generate new data

After training, sample new data like this:



Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from unobserved (latent) representation **z**

How can we train this?

Basic idea: **maximum likelihood**

If we had a dataset of (x, z) then train a *conditional generative model* p(x | z)

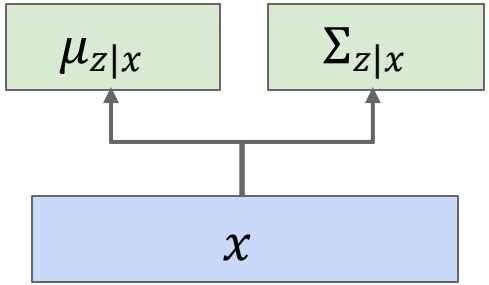
Variational Autoencoders

Jointly train **encoder** q and **decoder** p to maximize the **variational lower bound** on the data likelihood
Also called **Evidence Lower Bound** (**ELBo**)

$$\log p_\theta(x) \geq E_{z \sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$

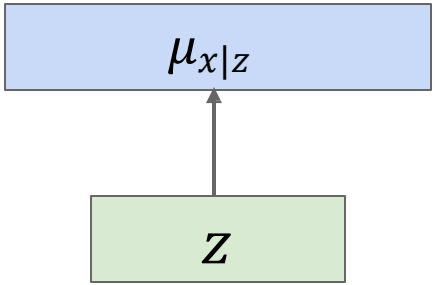
Encoder Network

$$q_\phi(z \mid x) = N(\mu_{z|x}, \Sigma_{z|x})$$



Decoder Network

$$p_\theta(x \mid z) = N(\mu_{x|z}, \sigma^2)$$



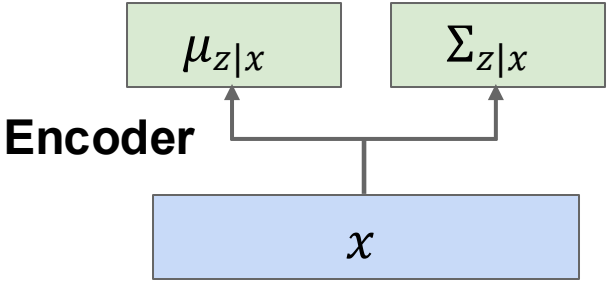
Variational Autoencoders: Training

Train by maximizing the
variational lower bound

$$E_{z\sim q_\phi(z|x)}[log\ p_\theta(x|z)] - \boxed{D_{KL}\left(q_\phi(z|x), p(z)\right)}$$

1. Run input data through **encoder** to get distribution over z
2. Prior loss: Encoder output should be unit Gaussian (zero mean, unit variance)

$$q_\phi(z \mid x) = N(\mu_{z|x}, \Sigma_{z|x})$$



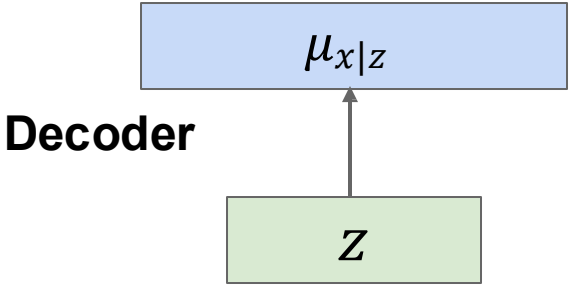
Variational Autoencoders: Training

Train by maximizing the
variational lower bound

$$\boxed{E_{z\sim q_\phi(z|x)}[log\ p_\theta(x|z)]} - D_{KL}\left(q_\phi(z|x), p(z)\right)$$

1. Run input data through **encoder** to get distribution over z
2. Prior loss: Encoder output should be unit Gaussian (zero mean, unit variance)
3. Sample z from encoder output $q_\phi(z \mid x)$ (Reparameterization trick)
4. Run z through **decoder** to get predicted data mean
5. Reconstruction loss: predicted mean should match x in L2

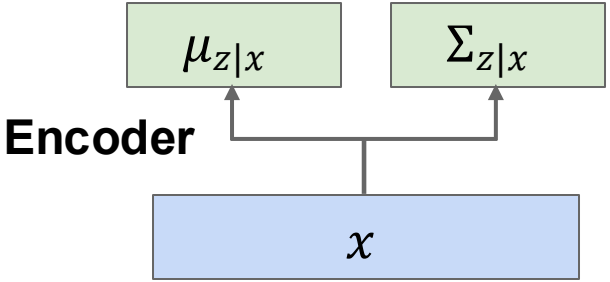
$$p_\theta(x \mid z) = N(\mu_{x|z}, \sigma^2)$$



$$\epsilon \sim N(0, I)$$

$$z = \epsilon \odot \Sigma_{z|x} + \mu_{z|x}$$

$$q_\phi(z \mid x) = N(\mu_{z|x}, \Sigma_{z|x})$$



Variational Autoencoders: Training

Train by maximizing the
variational lower bound

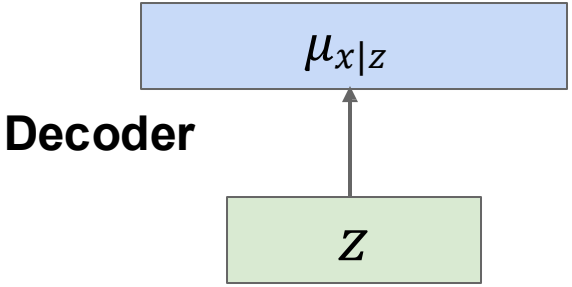
$$E_{z\sim q_\phi(z|x)}[log\ p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$

The loss terms fight against each other!

Reconstruction loss wants $\Sigma_{z|x} = 0$ and $\mu_{z|x}$ to be unique for each x, so decoder can deterministically reconstruct x

Prior loss wants $\Sigma_{z|x} = \mathbf{I}$ and $\mu_{z|x} = 0$ so encoder output is always a unit Gaussian

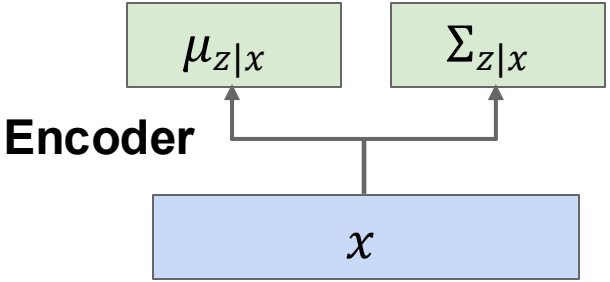
$$p_\theta(x \mid z) = N(\mu_{x|z}, \sigma^2)$$



$$\epsilon \sim N(0, I)$$

$$z = \epsilon \odot \Sigma_{z|x} + \mu_{z|x}$$

$$q_\phi(z \mid x) = N(\mu_{z|x}, \Sigma_{z|x})$$



Implementing VAE loss functions

```
recons_loss =F.mse_loss(recons, input)

kld_loss = torch.mean(
    -0.5 * torch.sum(
        1 + log_var - mu ** 2 - log_var.exp(),
        dim = 1
    ),
    dim = 0
)
```

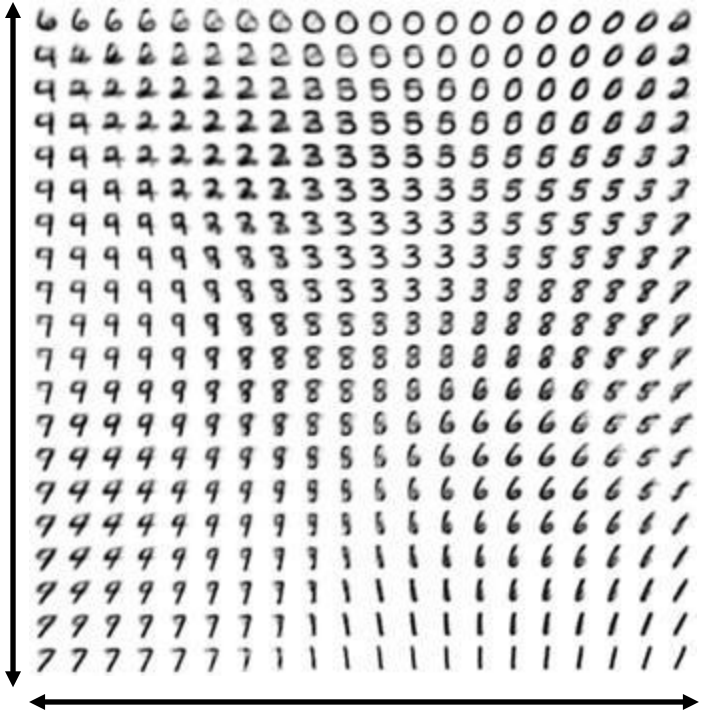
https://github.com/AntixK/PyTorch-VAE/blob/master/models/vanilla_vae.py

Variational Autoencoders: Disentangling

The diagonal prior on p(z) causes dimensions of z to be independent

"Disentangling factors of variation"

Vary $\mathbf{z_1}$



Vary $\mathbf{z_2}$

Kingma and Welling, Auto-Encoding Variational Beyes, ICLR 2014

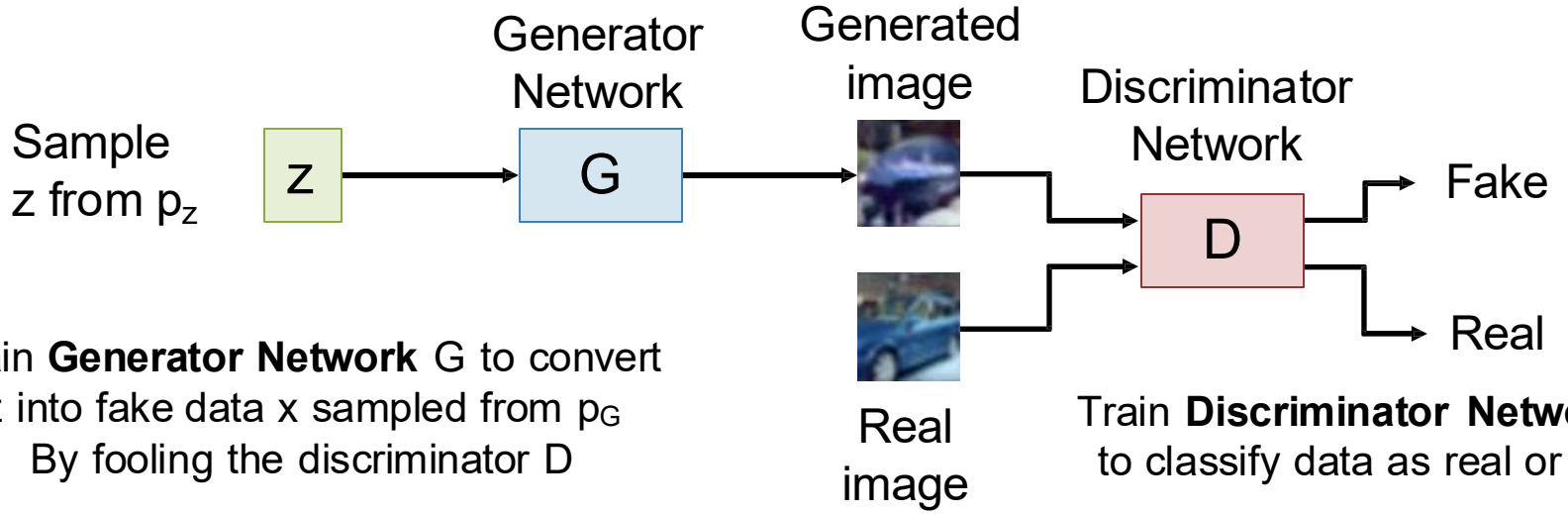
Generative Adversarial Networks (GANs)

Goodfellow et al, "Generative Adversarial Nets", NeurIPS 2014

Generative Adversarial Networks

Setup: Have data x_i drawn from distribution $p_{data}(x)$. Want to sample from p_{data}

Idea: Introduce a latent variable z with simple prior p(z) (e.g. unit Gaussian)
Sample $z \sim p(z)$ and pass to a **Generator Network** x = G(z)
Then x is a sample from the **Generator distribution** p_G. Want $p_G = p_{data}$!



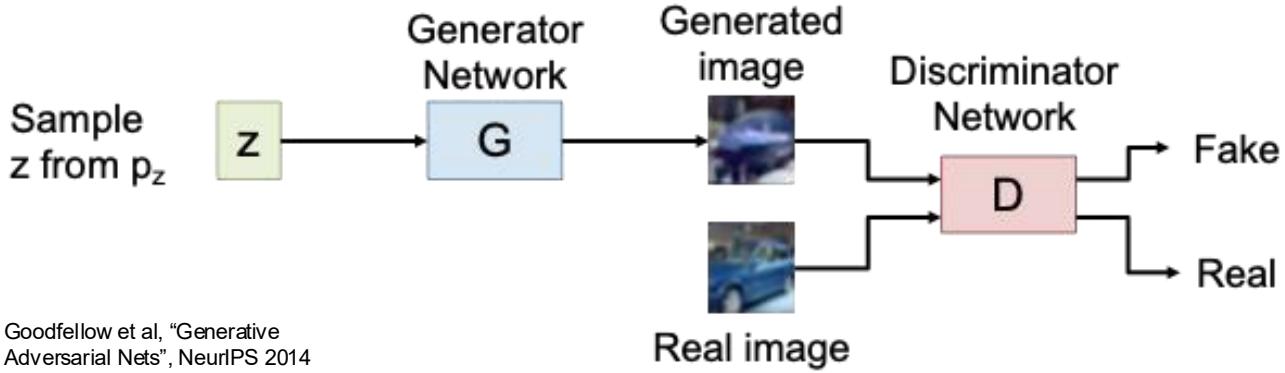
Train **Generator Network** G to convert z into fake data x sampled from p_G By fooling the discriminator D

Train **Discriminator Network** D to classify data as real or fake

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{G} \max_{D} \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$



D(x) = P(x is real)
D(x) = 0 => fake
D(x) = 1 => real

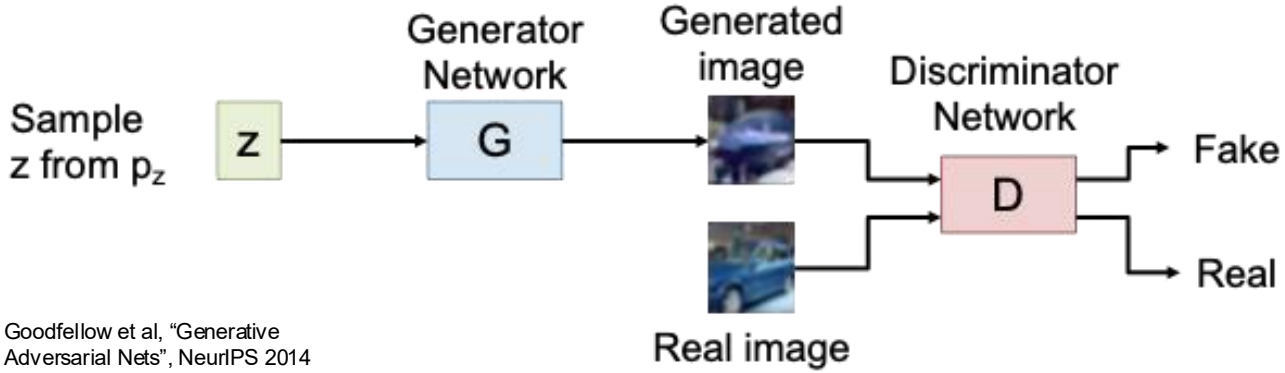
Goodfellow et al, "Generative Adversarial Nets", NeurIPS 2014

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Imagine fixing **G**

$$\min_{G} \max_{D} \left(\overbrace{E_{x \sim p_{data}} [\log D(x)]}^{\text{Discriminator wants D(x) = 1 for real data}} + \overbrace{E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right]}^{\text{Discriminator wants D(x) = 0 for fake data}} \right)$$



D(x) = P(x is real)
D(x) = 0 => fake
D(x) = 1 => real

Goodfellow et al, "Generative Adversarial Nets", NeurIPS 2014

Generative Adversarial Networks: Training Objective

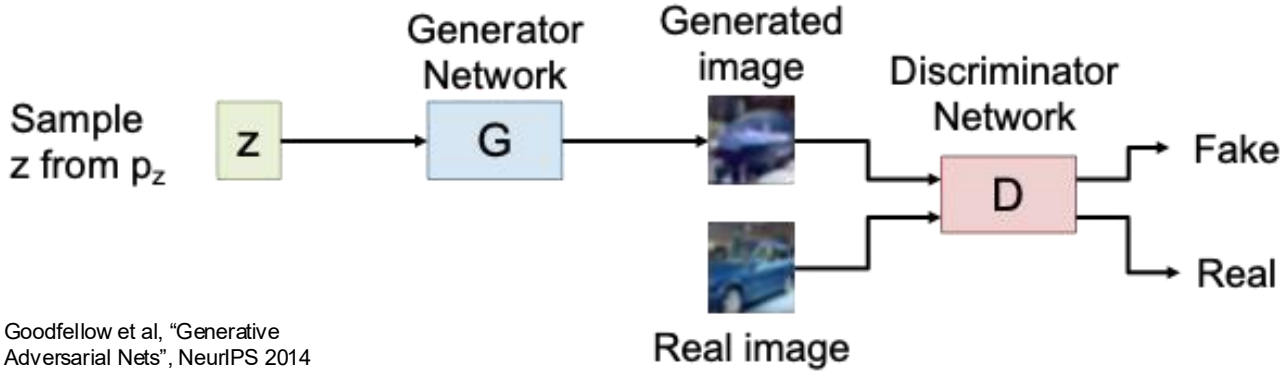
Jointly train generator G and discriminator D with a **minimax game**

Imagine fixing **D**

This term does not depend on G

Generator wants D(x) = 1 for fake data

$$\min_{G} \max_{D} \left(E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$



D(x) = P(x is real)
D(x) = 0 => fake
D(x) = 1 => real

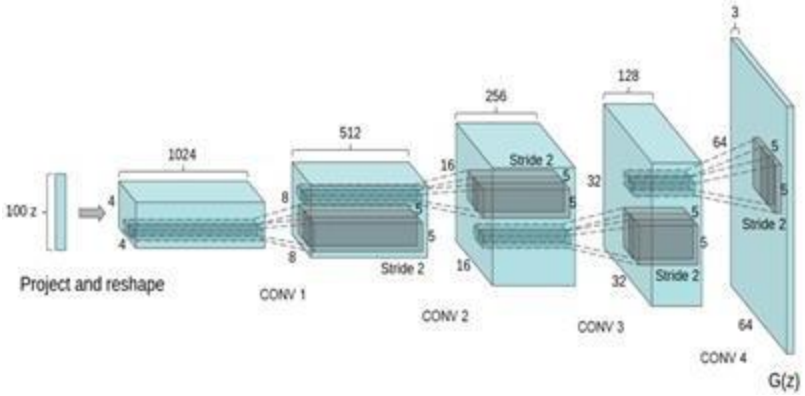
Goodfellow et al, "Generative Adversarial Nets", NeurIPS 2014

GAN Architectures: DC-GAN

Generator G and discriminator D are both neural networks

Usually CNNs … GANs fell out of favor before ViT became popular

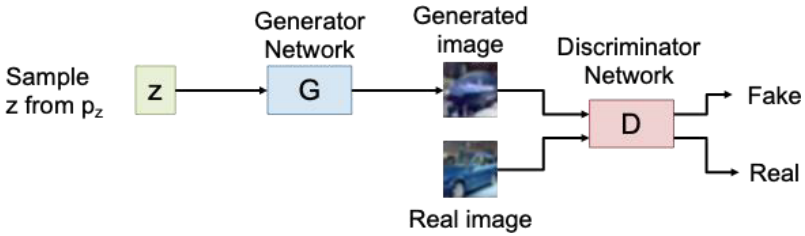
DC-GAN was the first GAN architecture that worked on non-toy data



Radford et al, ICLR 2016

Generative Adversarial Networks

Jointly train Generator and Discriminator with a minimax game



Pros:
- Simple formulation
- Very good image quality

Cons:
- No loss curve to look at
- Unstable training
- Hard to scale to big models + data

These were the go-to generative models from ~2016 – 2021

Diffusion Models

Sohl-Dickstein et al, “Deep Unsupervised Learning using noneuilibrium thermodynamics”, ICML 2015
Song and Ermon, “Generative modeling by estimnating gradients of the data distribution”, NaurIPS 2019
Ho et al, “Denoising Diffusion Probabalistic Models”, NeurIPS 2020
Song et al, “Score-Based Generative Modeling through Stochastic Differential Equations”, ICLR 2021
Song et al, “Denoising Diffusion Implicit Models”, ICLR 2021

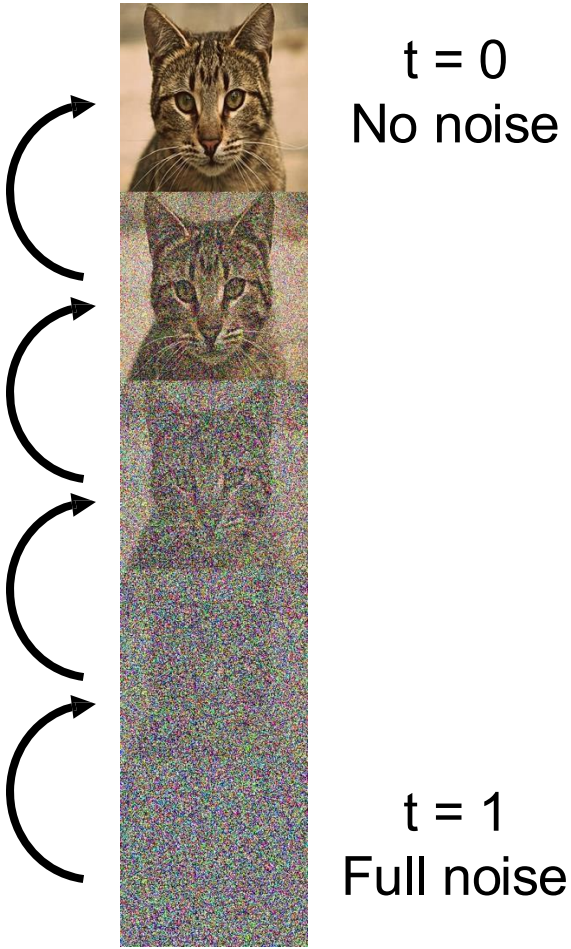
Diffusion Models: Intuition

Pick a **noise distribution** $z \sim p_{noise}$
(Usually unit Gaussian)

Consider data x corrupted under varying **noise levels** t to give noisy data x_t

Train a neural network to **remove a little bit of noise**: $f_\theta(x_t, t)$

At inference time, sample $x_1 \sim p_{noise}$ and apply f_θ many times in sequence to generate a noiseless sample x_0



Diffusion Models: Rectified Flow

Suppose we have a simple p_{noise} (e.g. Gaussian) and samples from p_{data}

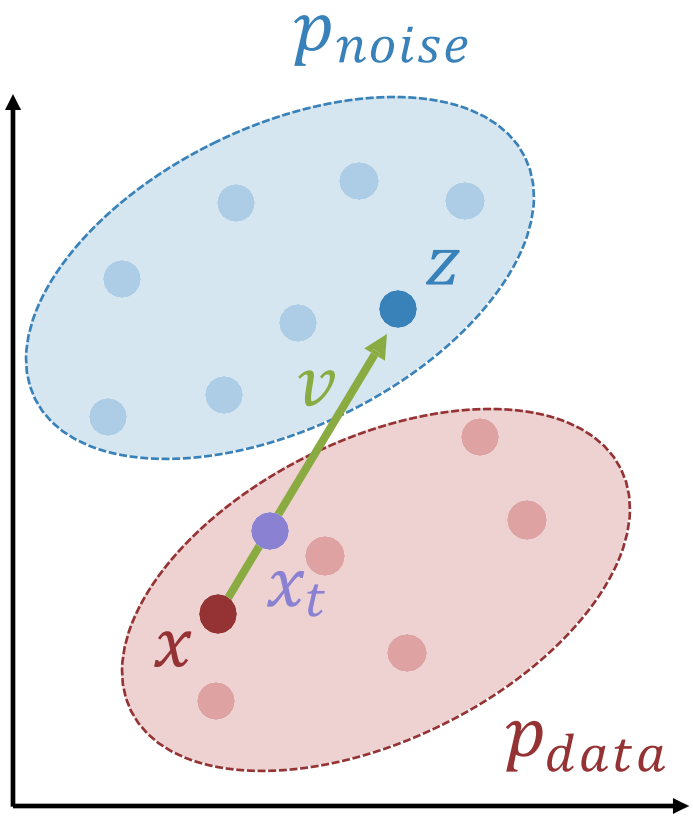
On each training iteration, sample:

$z \sim p_{noise} \quad x \sim p_{data} \quad t \sim Uniform[0,1]$

Set $x_t = (1-t)x + tz$, $v = z - x$

Train a neural network to predict v:

$$L = \|f_\theta(x_t, t) - v\|_2^2$$

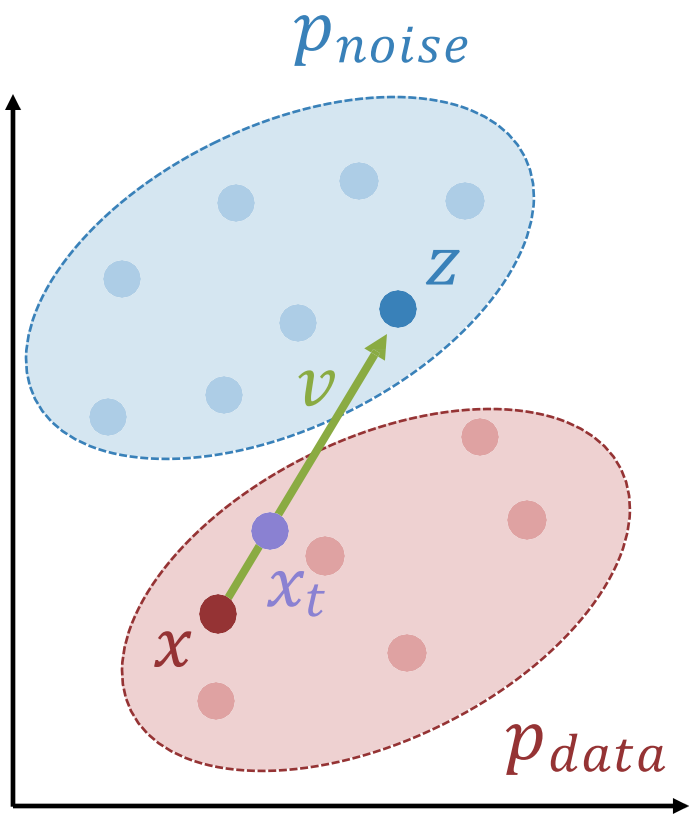


Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Diffusion Models: Rectified Flow

Core training loop is just
a few lines of code!

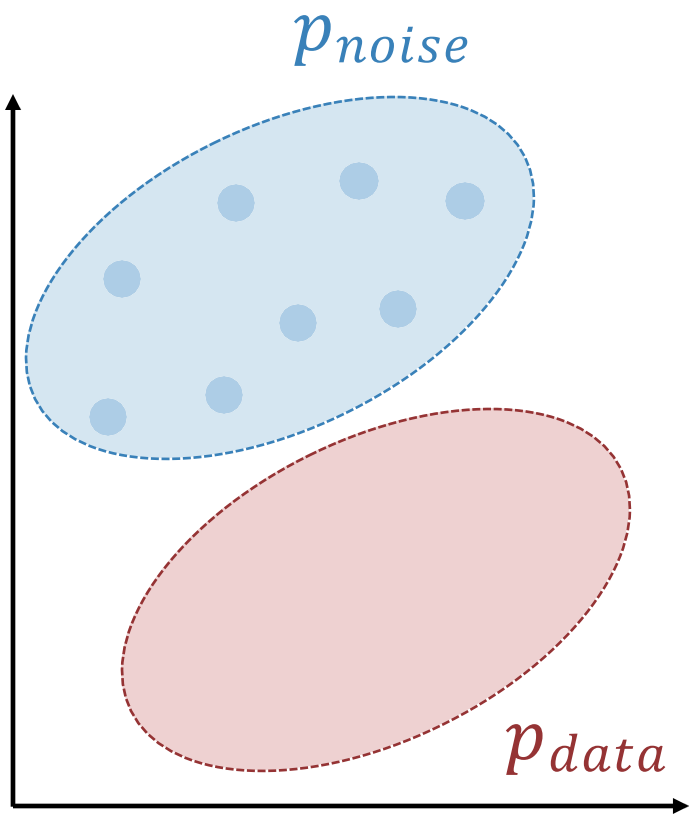
```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

Choose number of steps T (often T=50)

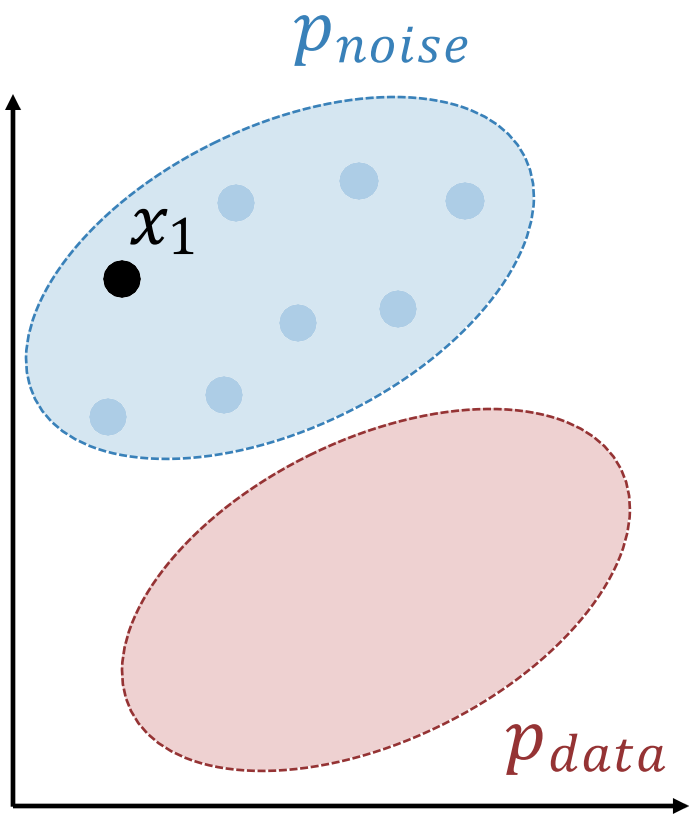


Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

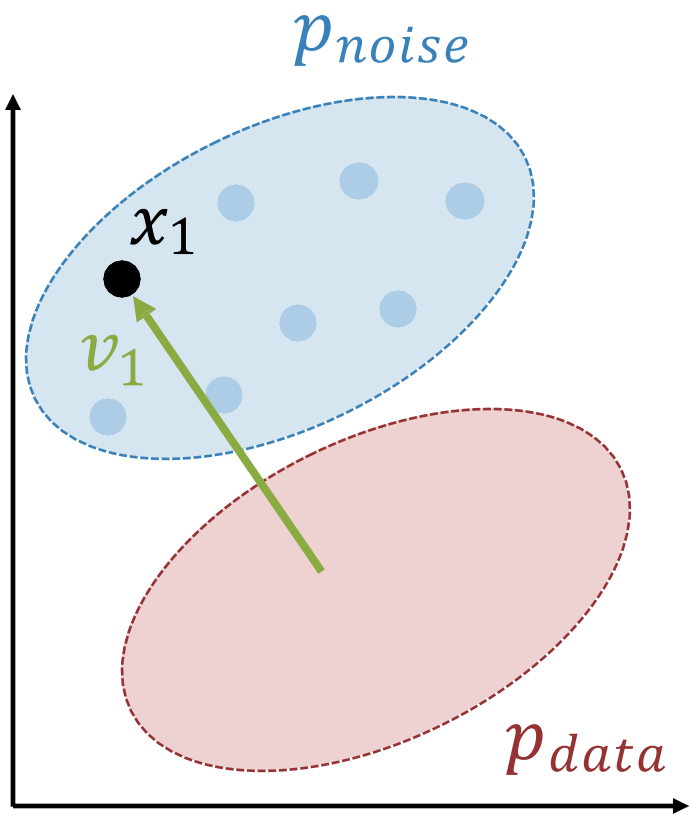
Rectified Flow: Sampling

Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

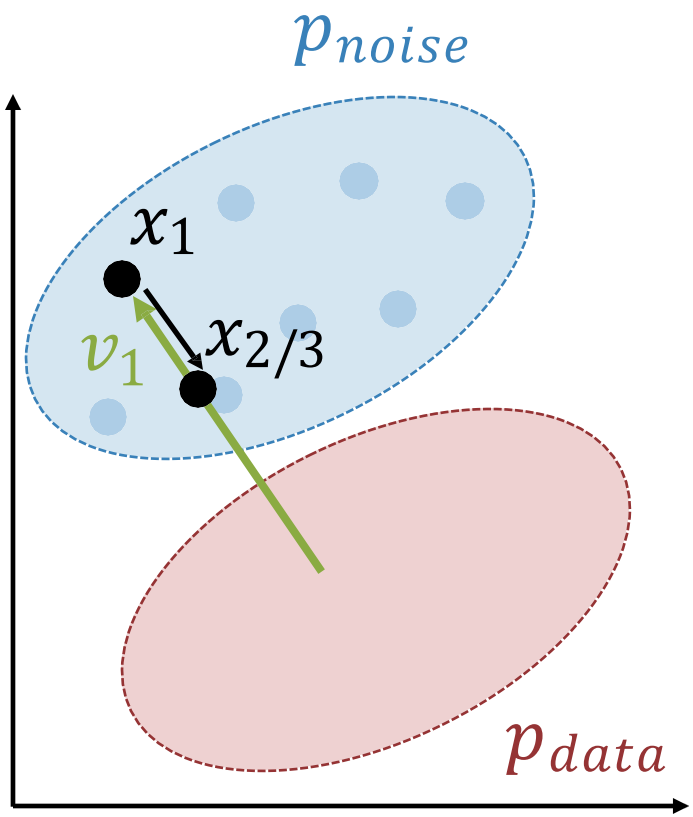
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

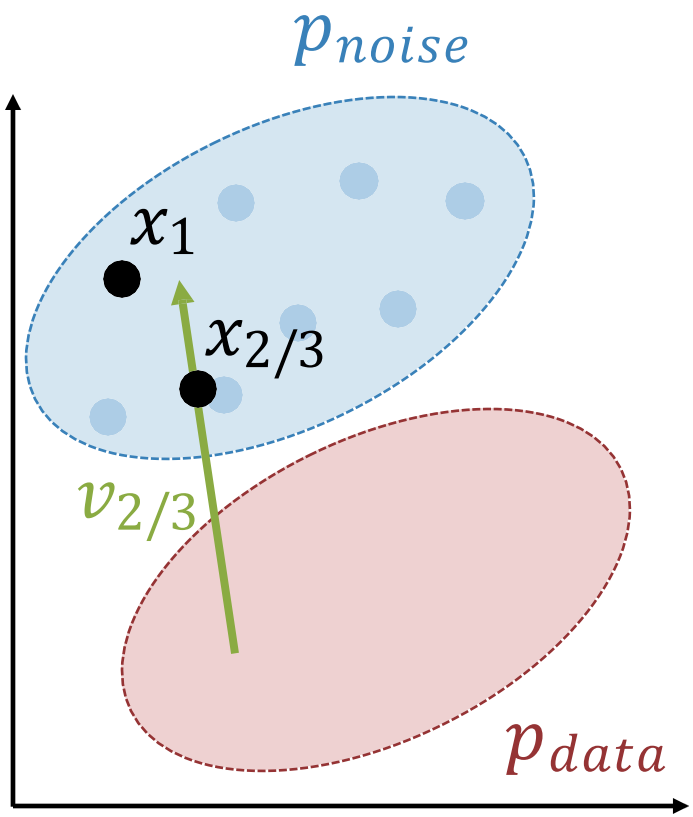
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

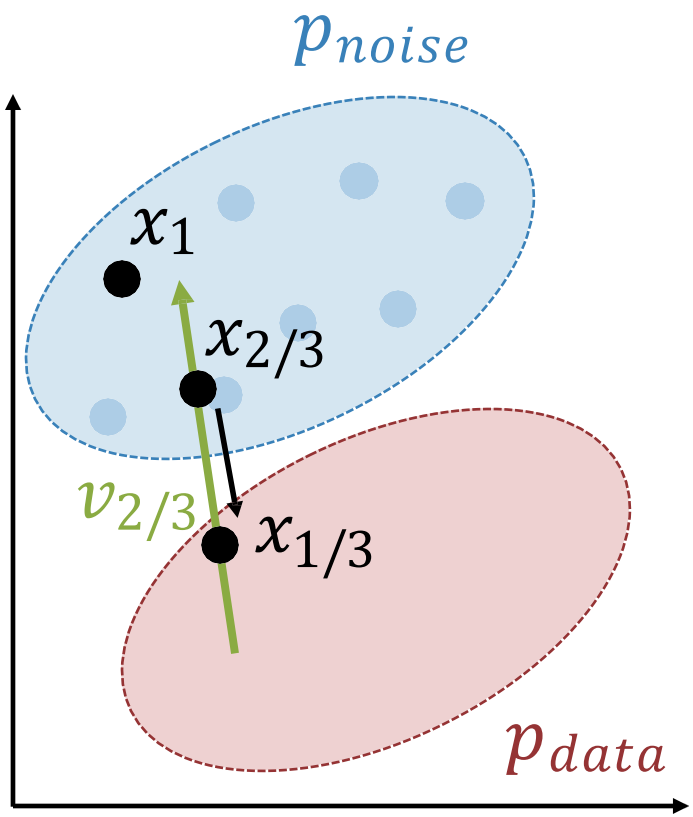
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

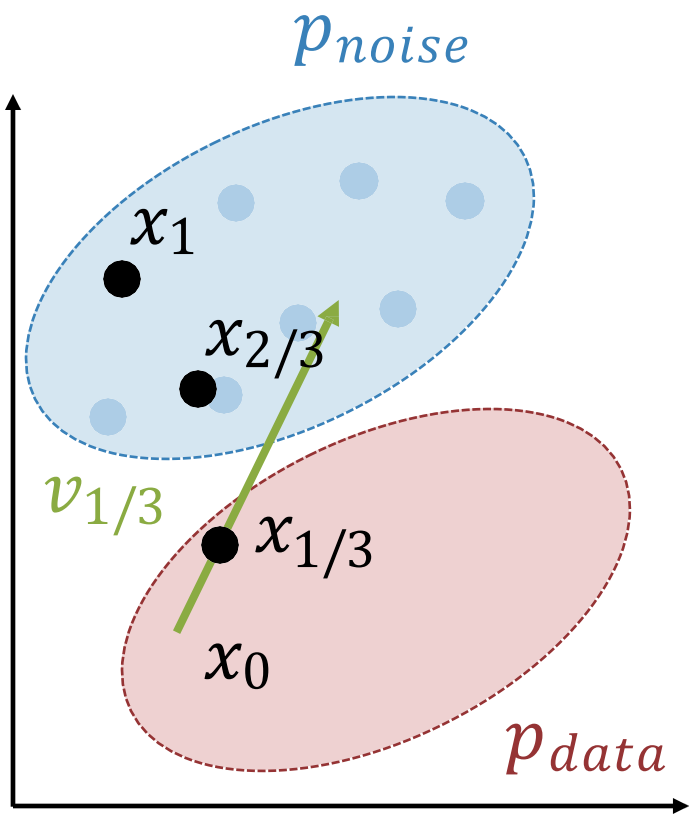
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Rectified Flow: Sampling

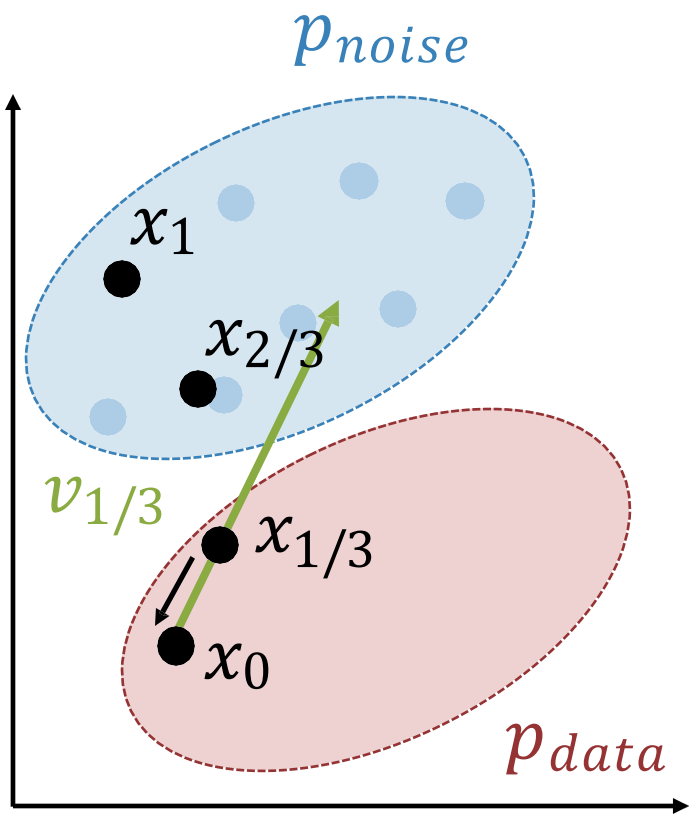
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Rectified Flow: Sampling

Choose number of steps T (often T=50)

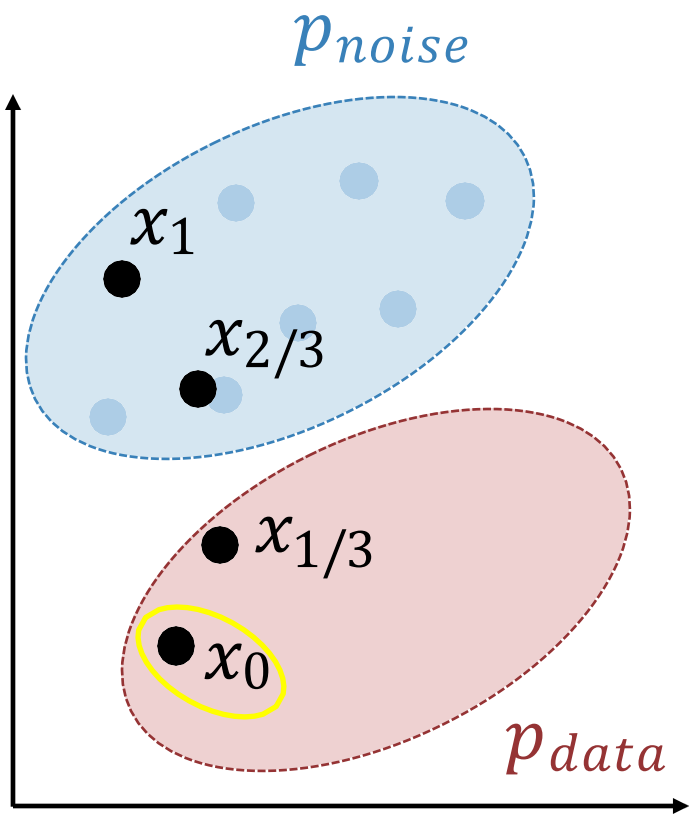
Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

  Evaluate $v_t = f_\theta(x_t, t)$

  Step $x = x - v_t/T$

Return x



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

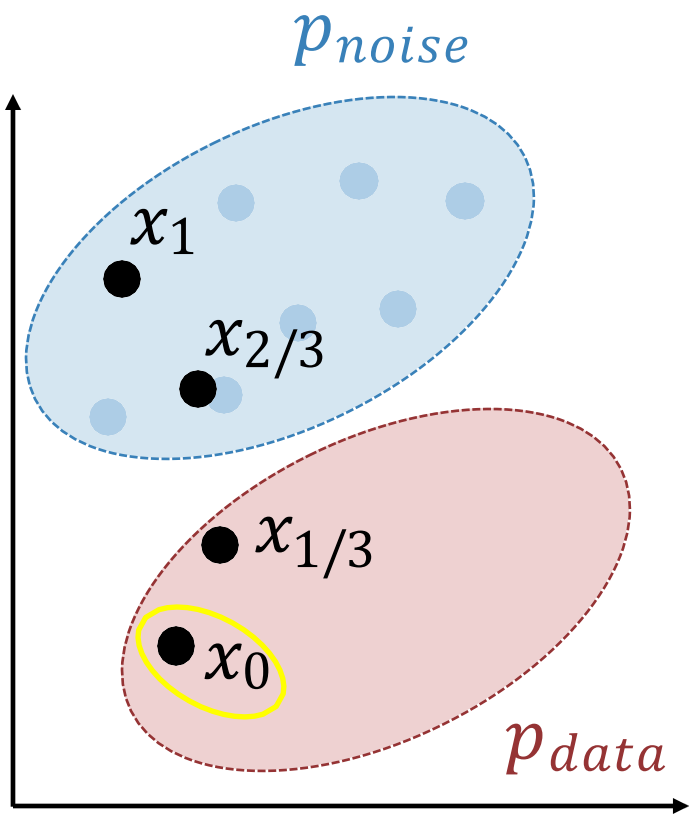
For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

  Evaluate $v_t = f_\theta(x_t, t)$

  Step $x = x - v_t/T$

Return x

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

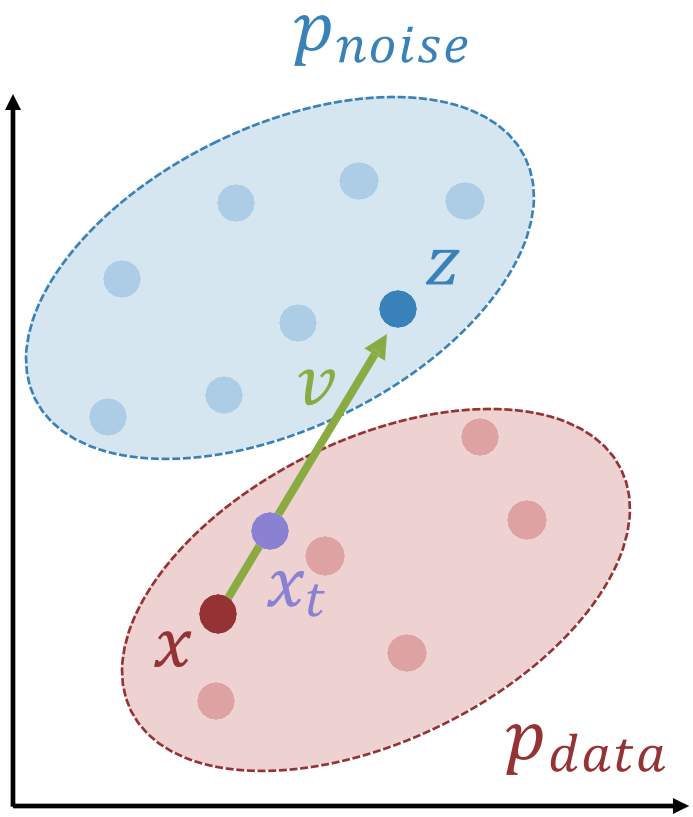
Rectified Flow: Summary

Training

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

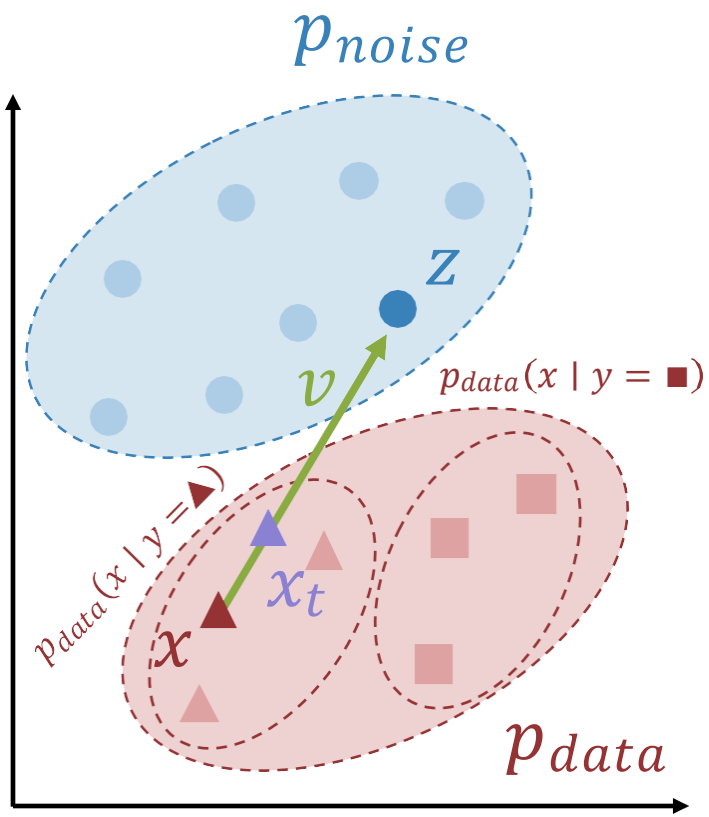
Conditional Rectified Flow

Training

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

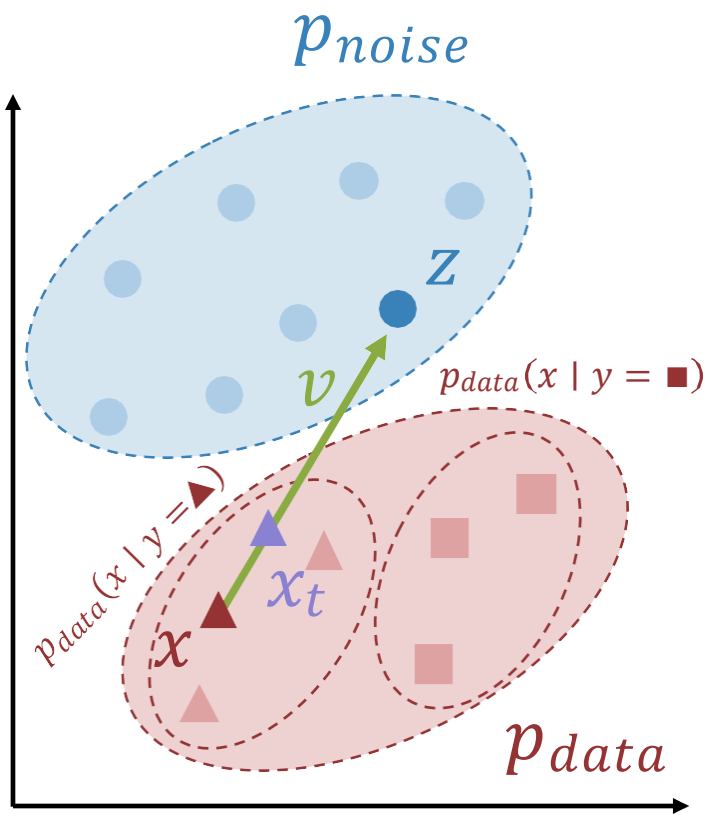
Conditional Rectified Flow

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

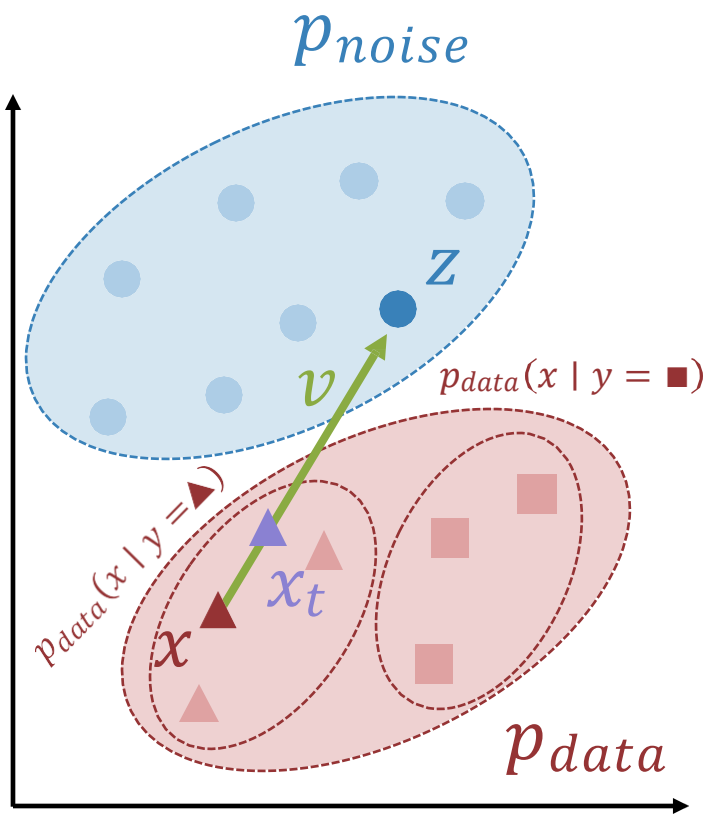
Conditional Rectified Flow

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, y, t)
    sample = sample - v / num_steps
```

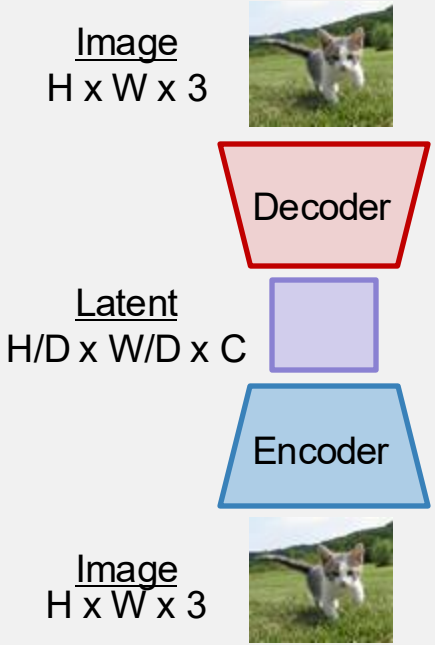


Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

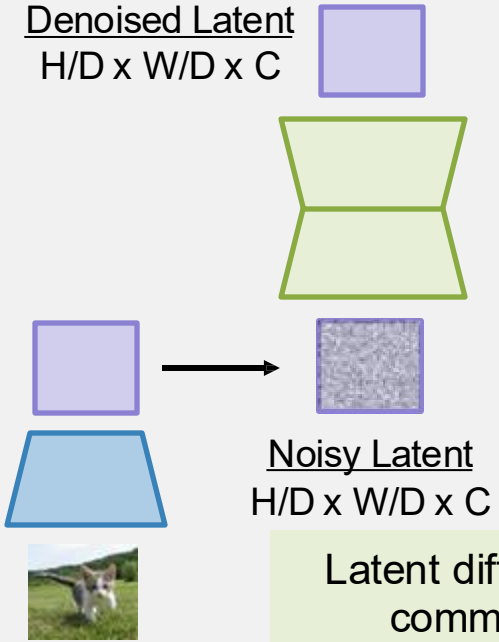
Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to convert images to **latents**



Train **diffusion model** to remove noise from **latents** (**Encoder** is frozen)

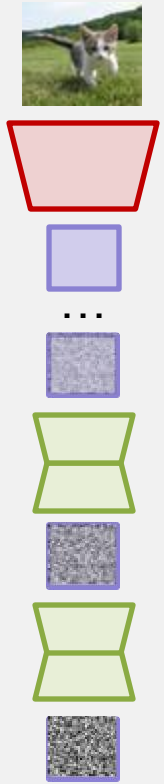


After training:

Sample random **latent**

Iteratively apply **diffusion model** to remove noise

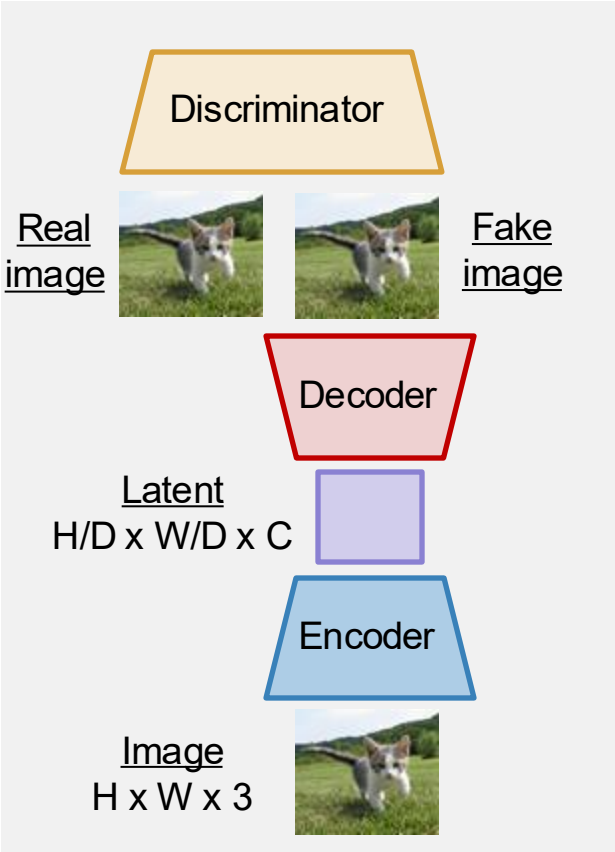
run **decoder** to get **image**



Latent diffusion is the most common form today

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

Latent Diffusion Models (LDMs)



How do we train the encoder+decoder?

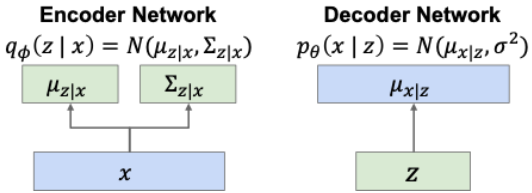
Solution: It's a VAE! Typically with very small KL prior weight

Problem: Decoder outputs often blurry

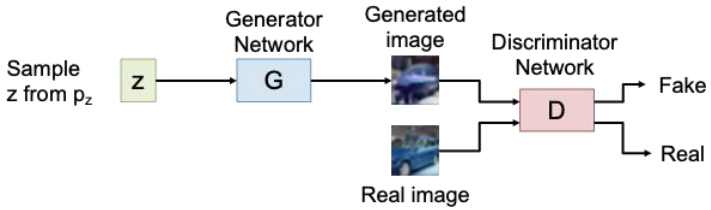
Solution: Add a discriminator!

Recall: VAE

$$\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$

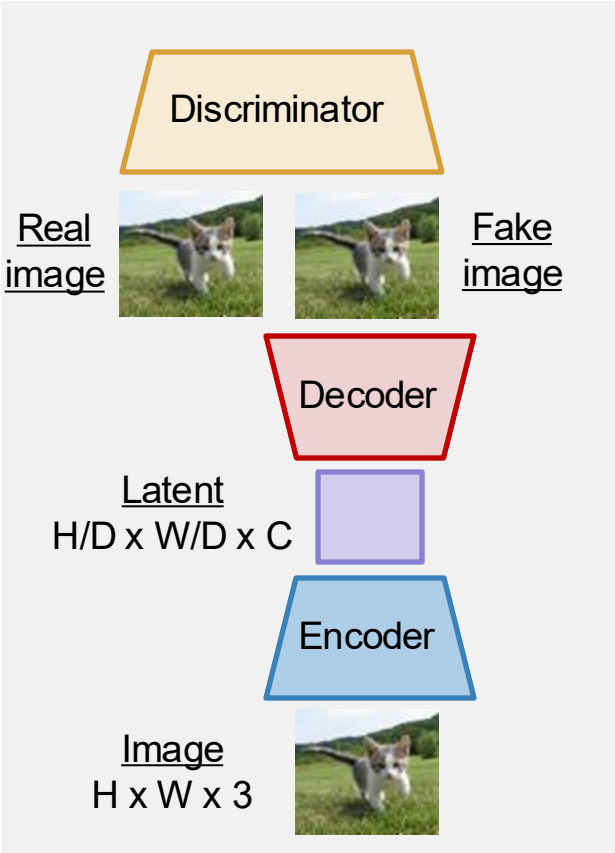


Recall: GAN

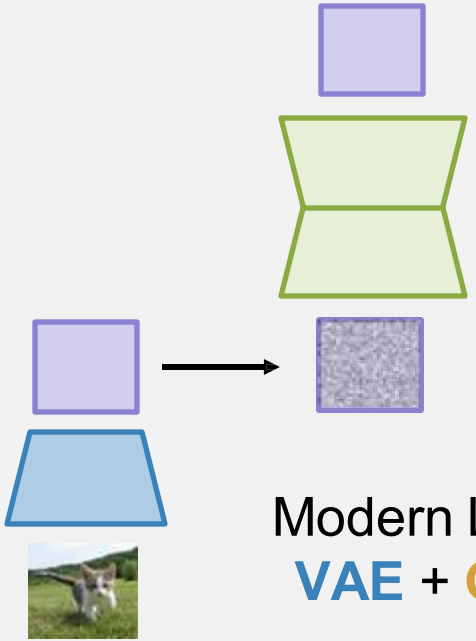


Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

Latent Diffusion Models (LDMs)



Train **diffusion model** to remove noise from **latents** (**Encoder** is frozen)

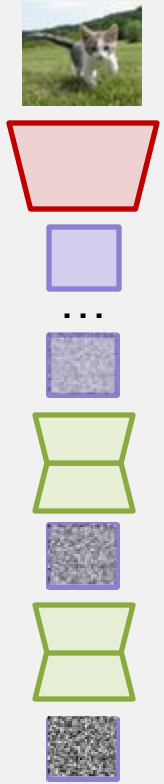


After training:

Sample random **latent**

Iteratively apply **diffusion model** to remove noise

run **decoder** to get **image**



Modern LDM pipelines use **VAE** + **GAN** + **diffusion**!

Text-to-Image

Diffusion timestep
(scalar)

Noisy latents
128 x 128 x 16

Text embeddings
D x L

Diffusion Transformer

Clean latents
128 x 128 x 16

Decoder

Output image
1024 x 1024 x 3

Pretrained text encoder
(e.g. T5, CLIP)

Text Prompt

A professional documentary photograph of a monkey shaking hands with a tiger in front of the Eiffel tower. The monkey is wearing a hat made out of bananas, and the tiger is standing on two legs and wearing a suit.

Example: FLUX.1 [dev]

Text Encoder: T5 + CLIP **Encoder/Decoder**: 8x8 downsampling **Diffusion model**: 12B parameter model 2x2 patchify => 64x64 = 1024 image tokens

https://github.com/black-forest-labs/flux